MIDTERM ESSAY

Course: Mining Massive Data Sets

Duration: 03 weeks

I. Formation

- The essay is conducted in groups of 02 – 03 students.
- Student groups conduct designated tasks and submit the essay by the deadline.

II. Datasets

Given **baskets.csv** file, consisting of shopping data, in which the first row is header and the remaining ones are records.

- **Member_number**: customer number
- **Date**: date in dd/mm/yyyy
- **itemDescription**: product name
- **year**: year
- **month**: month
- **day**: day
- **day_of_week**: day of week

For example,

Member_number	Date	itemDescription	year	month	day	day_of_week
1249	01/01/2014	citrus fruit	2014	1	1	2
1249	01/01/2014	coffee	2014	1	1	2
1381	01/01/2014	curd	2014	1	1	2
1381	01/01/2014	soda	2014	1	1	2
1440	01/01/2014	other vegetables	2014	1	1	2
1440	01/01/2014	yogurt	2014	1	1	2
1659	01/01/2014	specialty chocolate	2014	1	1	2
1659	01/01/2014	frozen vegetables	2014	1	1	2
1789	01/01/2014	hamburger meat	2014	1	1	2
1789	01/01/2014	candles	2014	1	1	2

baskets.csv (displayed in Google Colab)

III. Requirements

1) Task 1 (2.0 point(s)): Hadoop MapReduce

a) Fully-distributed Hadoop (1.0 point(s))

Students set up a Hadoop system in the fully-distributed mode and then compose a tutorial similar to Tutorial 01.

Requirements:

- Use Docker containers instead of Ubuntu virtual machines.
- There must be at least 05 nodes, including 01 Name Node, 01 Secondary Name Node, and 03 Data Nodes.
- Execute the Word Count job (source code provided in Tutorial 03) to verify your installation and report results, especially the monitor pages of Hadoop.
- Submit the tutorial as a docx file (editable).

b) Counting degrees (1.0 point(s))

- Generate an edge list of a directed graph with 100,000 vertices and 1000,000,000 edges (no self-loop). Copy the generated text file to HDFS. You are allowed to use an in-memory program.
- Implement a Hadoop MapReduce program (Java) to determine the in- and out-degrees of each vertex.
- Set up an experiment to contrast the running time with and without using combiners in the above program.

2) Task 2 (2.0 point(s)): RDDs vs DataFrames

a) RDDs (1.0 point(s))

- Use RDDs (do not use DataFrames) of PySpark to read the **baskets.csv** file. Then, implement, execute, save, and visualize results of the following functions.

Function	*Input*	*Output*	*Processing*
f1	The path to **baskets.csv**	Display results on the screen and save them to folder **f1**	Identify top 10 customers who bought the largest number of distinct items. Visualize the result using a bar chart.
f2	The path to **baskets.csv**	Display results on the screen and save them to folder **f2**	Determine the number of shopping baskets bought in each month.

			Visualize the result using a line chart.
f3	The path to **baskets.csv**	Display results on the screen and save them to folder **f3**	Discover the most frequent pair of items in each month. Print the result on the screen.
f4	The path to **baskets.csv**	Display results on the screen and save them to folder **f4**	Identify the customer who exhibits the most irregular purchasing behavior. Irregularity is defined as having the largest standard deviation of the number of items purchased per day. Print the result on the screen.

b) DataFrames (1.0 point(s))

- Re-implement task (a) using PySpark DataFrame.

3) Task 3 (3.0 point(s)): Frequent ItemSet Mining

a) Basket Discovery (1.0 point(s))

- Store given data set (**baskets.csv**) in HDFS and then implement a Hadoop MapReduce program (Java) to discover baskets (a basket is a set of items bought by 01 customer on 01 date).

b) PCY algorithm (2.0 point(s))

- Implement the PCY algorithm as Hadoop MapReduce jobs (Java) to identify frequent pairs given discovered baskets from Task 3a and a support threshold *s*. Describe, in details, the hashing function and bucket management.
- The algorithm is organized in form of OOP classes to support software deployment. Each pass is implemented in a single class in which there are two inner classes for Mapper and Reducer.
- Execute jobs in the virtual cluster of Task 1.
- Set up an experiment to contrast the running time with and without using combiners.
- 1.0 point for each pass of the algorithm.

4) Task 4 (2.0 point(s)): Association Rule Generation

- Use PySpark DataFrame to read baskets and frequent pairs of items from Task 3 and implement an association rule generator in form of a Python class as below.

- (**0.25 point(s)**) Constructor: input a data frame of baskets, a data frame of frequent pairs, a confidence threshold *c*, an interest threshold *i*.
- (**0.5 point(s)**) Method *generateRules*(): generate non-trivial rules and remove duplicates.
- (**0.5 point(s)**) Method *computeConfidence*(): compute confidence values of rules and filter rules based on the given threshold *c*.
- (**0.5 point(s)**) Method *computeInterest*(): compute interest values of rules and filter rules based on the given threshold *i*.
- (**0.25 point(s)**) Method *transform*(): take in sets of items, look up related rules, and then return recommended items on the right hand side of rules.

- In-memory computation is not allowed, distributed computation is required.

5) **Task 5 (1.0 point(s)): Report**

- Student groups compose the project report using the IEEE conference proceeding template.
- Recommended editor: Overleaf.
- Selective contents:
  - *Title*: the project title
  - *Authors*: group member's information, the lecturer is appended as the last author.
  - *Abstract*: summarize the project requirements, approaches, experimental results, and levels of completion.
  - Each following section presents a task in the project, with a meaningful and human-readable title. Briefly introduce the approach to tackle the problem and illustrate results with related figures/tables, etc.
  - "*Contributions*" section: individual tasks, individual completion levels (0%-100%).
  - "*Self-evaluation*" section: self-evaluate task completion and estimate scores.
  - "*Conclusion*" section: summarize the project requirements, approaches, experimental results, and levels of completion.
- References are in the IEEE format.

- Maximal length is 05 pages.

IV. Submission Notice

- Create a folder whose name is like **midterm_<Project group ID>_<Student ID>**:
  - **Source/:** consists of the project source code, each task is implemented in an individual sub-directory, preserving the outputs of all cells in ipynb files, output files as well.
  - **Report/:** report source (exported from Overleaf), **report.pdf** file.
- Compress the folder as a zip file and submit by the deadline.

V. Policy

- **Student groups submitting late get 0.0 points for each member.**
- **Copying source code on the internet/other students, sharing your work with other groups, etc., cause 0.0 points for all related groups.**
- **If there exist any signs of illegal copying or sharing of the assignment, then extra interviews are conducted to verify student groups' work.**
- **Evaluation scores of individual tasks are only recorded if and only if the student group give a reasonable presentation and justification to avoid cheating by AI tools, rental of doing the project, imbalance contributions, missing discussing, cooperating of group members in the project, etc.**
- **AI tools are forbidden in this project. If any abnormal symbols generated by AI tools appear in the source code files, the group will receive a score of 0.0 for the corresponding task.**

-- THE END --